FHE BUIDL MUMBAI · ANALYTICS TRACK · 2026

fheENV

Your `.env`, encrypted. Not even us.

Fully Homomorphic Encryption | Fhenix CoFHE | Next.js + wagmi | CLI + Web + Contract

Hardhat · viem · Solidity 0.8.25 · IPFS / Pinata · AES-256-GCM

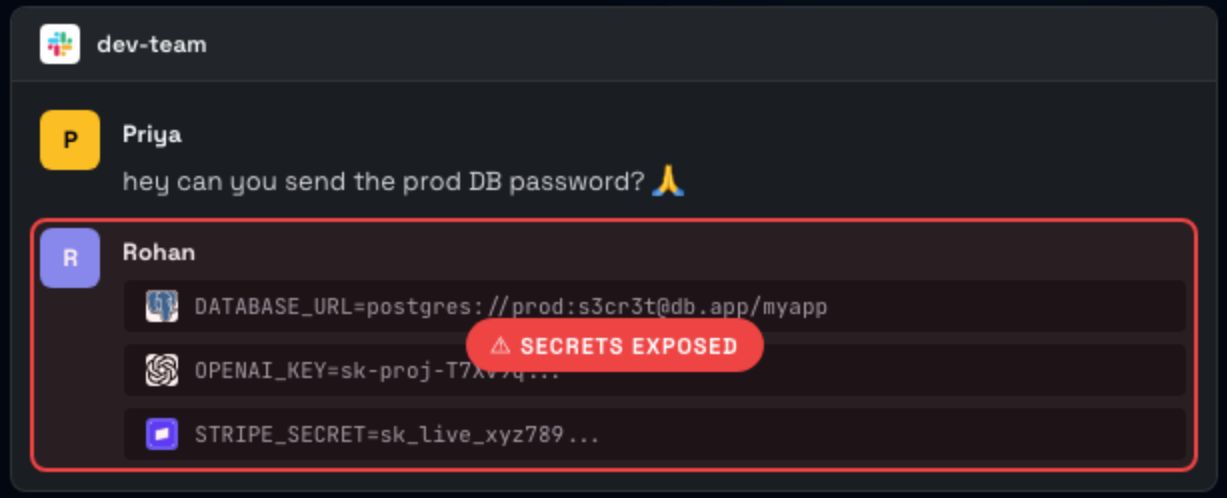
▲ THE REALITY

You've done this. Today.

Every dev team shares secrets on Slack.
That message lives on someone else's server — forever.

80%
of breaches start with a stolen credential

642+
credential breaches in 2025 alone



Now lives on Slack's servers · backups · forever.

✗ THE ILLUSION OF SECURITY

Same problem. Different server.

Doppler

Centralized server holds your keys in plaintext. $20M raised. Still reads your secrets.

⚠ Doppler can read your keys

HashiCorp Vault

Admin has full access. $6.4B IBM acquisition. Complex self-host.

⚠ Admin has full plaintext access

AWS Secrets

US servers. $0.40/secret/month. Subject to government subpoena.

⚠ US gov can subpoena it

All three require you to trust a company — that trust is your attack surface.

🔒 THE SOLUTION

Zero-trust secrets. Mathematically.

TODAY

📄 .env plaintext

↓

🖥️ Central server reads everything

↓

👥 Team

"Trust us" model

vs

FHEENV

📄 .env (your device only)

↓

🔒 AES-256 encrypt → IPFS blob

↓

⚙ AES key → 2× euint128 → FHE on-chain

↓

👛 Wallet decrypts via Threshold Network

Mathematically impossible to read

⚙ ARCHITECTURE

AES + FHE Envelope

01 **Generate AES-256 key**
Random per rotation. Never stored anywhere.

02 **AES-GCM encrypt → IPFS**
Blob is gibberish without the key. CID stored on-chain.

03 **Split AES key → 2× euint128 on-chain**
FHE-encrypted. No node can read it.

04 **FHE.allow() per wallet**
Threshold Network decrypts only to authorized address.

WHERE DATA LIVES

LOCATION	DATA	READABLE?
On-chain	2× euint128 AES handles	✗ FHE encrypted
IPFS	AES-encrypted .env blob	✗ Gibberish
Fhenix nodes	FHE computation	✗ Encrypted
Your device	Plaintext .env	✓ Only you

One FHE decrypt per pull
Whole .env uses one AES key → constant FHE overhead regardless of secret count.

⚡ WHY FHE

Other primitives can't do this.

✗ ZK Proofs
Prove "I know X" — can't selectively reveal encrypted data to different parties.

✗ AES / RSA alone
Someone must hold the decryption key — that someone is your attack surface.

✓ FHE (Fhenix CoFHE)
Key stored as ciphertext. `FHE.allow()` grants per-wallet decrypt. Threshold Network — no single party reconstructs.

OPERATIONS USED

`FHE.asEuint128()` — store AES key chunk
`FHE.allow(handle, addr)` — grant decrypt
`FHE.allowThis()` — re-grant on rotation

fheENVRegistry.sol

```
// AES key → two FHE-encrypted uint128
euint128 hi = FHE.asEuint128(inHigh);
euint128 lo = FHE.asEuint128(inLow);

FHE.allowThis(hi); FHE.allowThis(lo);
FHE.allow(hi, msg.sender);

// Grant teammate decrypt
FHE.allow(env.aesKeyHigh, member);
FHE.allow(env.aesKeyLow,  member);

// Revoke = rotate → new handles, old abandoned
updateEnvironment(id, name,
  newHi, newLo, newCid,
  expectedVersion // optimistic lock
);

// Batch grant up to 100 in one tx
batchGrantAccess(id, name, address[] members);
```

📦 WHAT WE BUILT

Three components. Fully working.

Smart Contract
Solidity 0.8.25 · Sepolia

- → createProject / addOwner
- → updateEnvironment + optimistic lock
- → grantAccess / batchGrantAccess
- → revokeAccess + mandatory rotate

Deployed · Sepolia

Web Dashboard
Next.js 14 · wagmi · @cofhe/sdk

- → Wallet auth — no email/password
- → Push: AES encrypt → IPFS
- → Pull: FHE decrypt → view secrets
- → Team: grant / revoke + audit log

Live App

CLI Tool
Node.js · npm installable

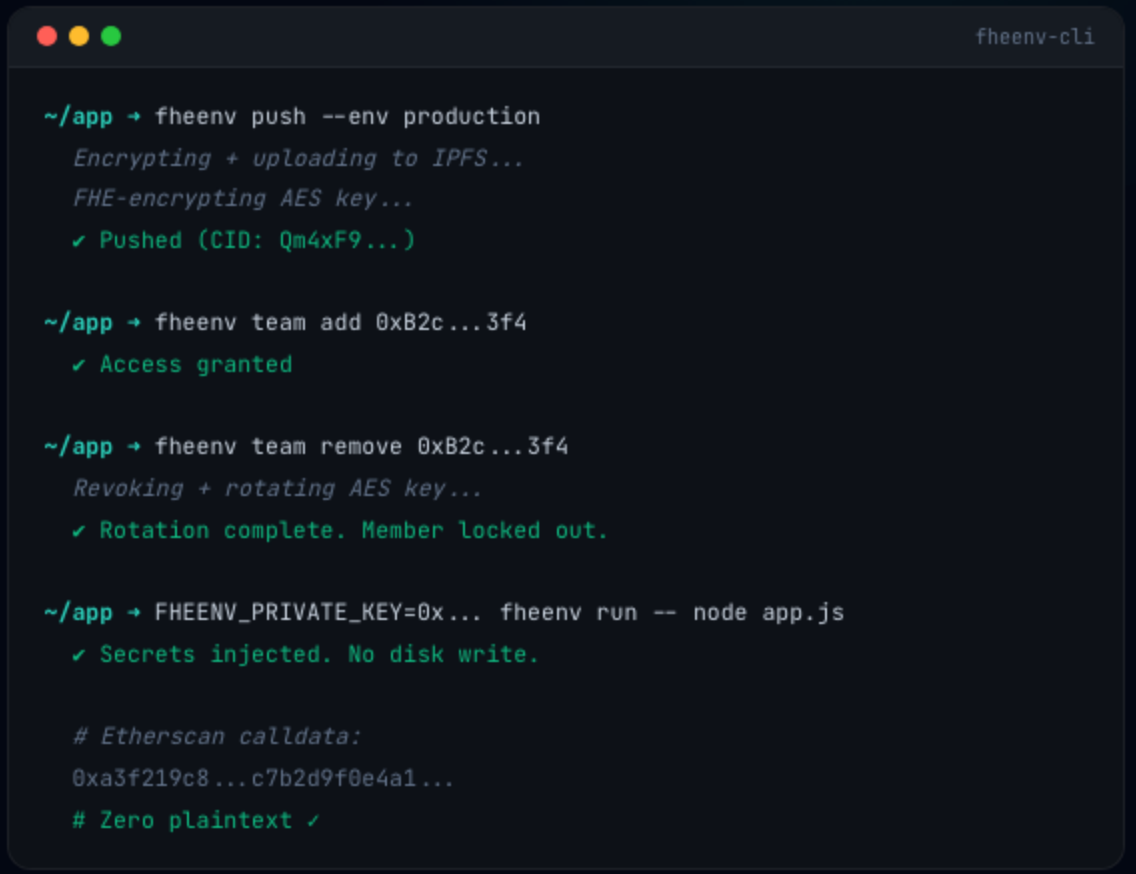
- → fheenv push / pull / run
- → fheenv rotate (new AES + re-grant)
- → fheenv team add / remove
- → FHEENV_PRIVATE_KEY for CI/CD

npm i -g fheenv

▸ DEMO FLOW

What judges will see.

1. **Push (Wallet A)**
   Paste .env → encrypt + upload → FHE-encrypt key halves → tx submitted.
2. **Inspect Etherscan**
   Zero readable secrets — only FHE-encrypted bytes visible.
3. **Grant (Wallet A → Wallet B)**
   FHE.allow() for both key halves. One transaction.
4. **Pull (Wallet B)**
   Threshold decrypts AES key locally → secrets appear. ✓
5. **Revoke + Rotate**
   New AES key → new FHE handles → Wallet B locked out cryptographically.



📊 ANALYTICS TRACK

Purpose-built for this track.

"Autonomous systems operating on encrypted information while maintaining security and user privacy"

TRACK CRITERIA	FHEENV
Encrypted information operations	AES-256 + FHE — never plaintext on any server
Autonomous systems	CI/CD pulls secrets autonomously via `FHEENV_PRIVATE_KEY`
Security & user privacy	Cryptographic access control + on-chain audit log
Fhenix FHE use case	`euint128`, `FHE.allow()`, rotation-enforced revocation

fheENV is the **secrets layer for the encrypted web** — foundational infrastructure for any app that handles credentials without a centralized trust point.

MARKET

$2.3B market. Zero FHE players.

$2.3B
Market size 2025

$8.9B
Projected 2030

$6.4B
HashiCorp → IBM

THE GAP

Doppler · Vault · AWS — all readable by operator

fheENV — mathematically private. Free.

INDIA OPPORTUNITY

5M+
Software developers — fastest-growing dev market

DPDP Act 2023
India's new data protection law demands demonstrable security. fheENV delivers cryptographic proof — not policy.

Startup Fit
No subscription. No vendor trust. No lock-in. Free infra.

🏆 JUDGING

Built for this track.

CRITERIA	EVIDENCE
Problem Statement	Universal dev problem. Backed by breach stats. Clear villain: centralized trust.
Fhenix FHE Use Case	FHE is architecturally irreplaceable — `euint128`, `FHE.allow()`, Threshold Network. No substitute exists.
MVP / Demo Video	Live: push → Etherscan shows zero plaintext → wallet B decrypts → rotation locks out.
E2E Deployment	Contract on Sepolia. Web app live. CLI on npm. IPFS via Pinata.

🔒 FHEENV · FHE BUIDL MUMBAI 2026

fheENV

The Doppler alternative where **not even we** can see your secrets.

⚙ Fhenix CoFHE	📄 Solidity + Hardhat	🖥 Next.js + wagmi
⌨ Node.js CLI	☁ IPFS / Pinata	🔑 AES-256-GCM

GitHub: [repo link] Demo: [live URL]

> "Every dev in this room has a secret on Slack right now. fheENV means that never has to happen again."